Person 4: `attacks.py` (security testing with an attack model)

Your job in one line: the brief's point 4, "test the security of your protocol by applying any attack model". You define what the attacker can do, run 13 attacks plus 1 baseline, and show that each one is stopped and *which line* stops it.

Run your demo: `python attacks.py` (or `pytest attacks.py`)

---

1. What to say (about 3 minutes)

> "We use the **Dolev-Yao attack model**, the standard one for protocols. The attacker **owns the radio channel**: it can read, drop, replay, modify and inject any message. It can run its own rogue drones and even its own fake ground station. What it *can't* do is break the maths (AES, SHA-256, Ed25519, X25519) or read an honest drone's private key.
>
> Each test in `attacks.py` is one attack. If the protocol rejects it with an `AuthError`, the test passes. If the attack gets through, the test fails loudly. We have 13 attacks plus 1 honest baseline: replay, man-in-the-middle, impersonation, stolen certificate, revoked or expired drones, tampering, and forged confirmation. All are blocked."

Then run `python attacks.py` and point to the table below for 2 or 3 attacks.

---

2. Libraries in this file

Import	Where it's from	Why we need it
`copy`	stdlib	`deepcopy` so editing a certificate doesn't touch the original
`time`	stdlib	Make an already-expired timestamp
`X25519PrivateKey`	`cryptography`	The attacker generates its *own* DH key for the MITM attacks
`from gcs import GCS, AuthError, pub_hex`	Person 1	Make ground stations (also a fake one), detect rejections
`from drone import Drone, handshake`	Person 2	The protocol under attack

`pytest` is optional. It finds every `test_*` function automatically. Plain `python attacks.py` works without it.

---

3. Line by line, the helpers

```
def setup():
    gcs = GCS()
    return gcs, Drone("A", gcs), Drone("B", gcs)
```

A fresh world for every test: one GCS and two registered honest drones. Tests never share state.

```
def blocked(attack, reason):
    try:
        attack()
    except AuthError as e:
        assert reason in str(e), f"expected '{reason}', got '{e}'"
        return
    raise AssertionError("attack succeeded!")
```

Runs the attack, passed as a `lambda` (a function we call later).

- It raises `AuthError` with the expected reason, so the attack was **blocked by the right check**.
- It raises `AuthError` with a *different* reason, so the test fails. That catches "blocked, but by accident".
- It raises nothing, so **"attack succeeded!"** and the test fails.

```
if __name__ == "__main__":
    tests = [(name, fn) for name, fn in globals().items() if name.startswith("test_")]
```

Finds every `test_` function in the file and runs it in order, printing the docstring as the description.

---

4. Every attack: what it does, and which line stops it

Test	What the attacker does	Stopped by (file → check)	Lecture attack type
`test_honest_run`	nothing (baseline)	both drones get the same key	—
`test_replay_handshake`	records M1 and sends it again	`drone.py` `_check`: `msg["n"] in self._seen_nonces`	Replay
`test_replay_data_packet`	re-sends a recorded "land now" packet	`session.py` `open`: `seq <= self.recv_seq`	Replay
`test_stale_timestamp`	sends an M1 from 2 minutes ago	`drone.py` `_check`: `MAX_SKEW`	Replay
`test_mitm_swaps_dh_key`	replaces A's DH key in M1 with its own	`drone.py` `_check`: `verify_sig` fails	Man-in-the-middle
`test_mitm_on_response`	replaces B's DH key in M2	`drone.py` `_check` on A's side: `verify_sig`	Man-in-the-middle

Test	What the attacker does	Stopped by (file → check)	Lecture attack type
`test_impersonation_fake_cert`	rogue drone with a cert from a fake GCS	`gcs.py` `verify_certificate`: GCS signature	Impersonation
`test_stolen_cert_without_key`	shows A's real cert, signs with its own key	`drone.py` `_check`: `verify_sig`	Impersonation
`test_revoked_drone`	a captured drone keeps talking	`gcs.py` `verify_certificate`: CRL	Insider / authorization
`test_expired_cert`	edits the expiry on a certificate	`gcs.py` `verify_certificate`: signature breaks	Forgery
`test_misdirected_message`	delivers B's message to C	`drone.py` `_check`: `to` field	Redirection
`test_tampered_ciphertext`	flips one bit of a data packet	`session.py` `open`: AES-GCM `InvalidTag`	**Chosen-ciphertext**
`test_forged_key_confirmation`	sends M3 with a fake tag	`session.py` `check_tag`	MAC forgery
`test_fresh_keys_per_session`	(property) two sessions, same drones	different keys, because of ephemeral DH	Forward secrecy

How some attacks are coded

```
m1["eph"] = pub_hex(X25519PrivateKey.generate())   # MITM: attacker swaps in its own DH key
```

```
rogue = Drone("A", GCS())                          # impersonation: registered with a DIFFERENT GCS
```

```
eve.id, eve.cert = "A", a.cert                      # stolen cert: claim to be A with A's cert,
                                                    # but eve signs with her own private key
```

```
ct[0] ^= 1                                          # tamper: XOR flips the lowest bit of byte 0
```

---

5. Lecture-plan topics (CS428) and how to say them

"Overview of Types of Attacks" from the lecture plan, mapped to our protocol:

Attack type (lecture)	Attacker has	Why our protocol survives it
Ciphertext-only attack	only ciphertexts sniffed off the radio	AES-256-GCM ciphertexts are **indistinguishable** from random, so they reveal nothing
Known-plaintext attack	some plaintext/ciphertext pairs (telemetry has a known format like `{"lat":...}`)	AES is a pseudorandom permutation, so known pairs don't reveal the key; every packet uses a new nonce

Attack type (lecture)	Attacker has	Why our protocol survives it
Chosen-plaintext attack	can get chosen messages encrypted	nonce-based encryption is **CPA-secure**: same message → different ciphertext
Chosen-ciphertext attack	can submit modified ciphertexts	**authenticated encryption** rejects every modified ciphertext → `test_tampered_ciphertext`

Other topics you can name:

- **Computational Security**: "Our attacker is limited to realistic computation. The tests show the *protocol logic* holds and the maths is trusted."
- **Security against Chosen-Ciphertext Attacks / CCA-Secure Encryption**: same as the last row above.
- **Application area: Vehicular Network**: "Drones are a flying vehicular network, with the same threats as V2V."

---

6. Likely questions

- **Do passing tests *prove* the protocol is secure?** No. They show the defences work against these specific attacks. A formal proof would use a tool like ProVerif or Tamarin, or BAN logic. That's our future work, and saying so honestly scores well.
- **Why Dolev-Yao?** A radio link is broadcast. Anyone with an antenna can listen, jam, record and transmit. Assuming the attacker controls the channel is the realistic worst case.
- **What attacks are *not* covered?** Physically extracting keys from a captured drone (we limit the damage with revocation and expiry), jamming and denial of service (no crypto can stop jamming), and compromise of the GCS key itself (fix: secret sharing / threshold signing, also a lecture topic).
- **Why check the error *reason* and not just "an error happened"?** So we know the *right* defence fired. An attack blocked by the wrong check could mean the intended check is broken.